

SetTimeout and setInterval

setTimeout() — Run ONCE After Delay

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Executed After 2s");  
}, 2000);
```

```
console.log("End");
```

Output:

Start

End

Executed After 2s

Delay Is Minimum Time, Not Exact Time

If main thread is busy → it waits MORE.

```
setTimeout(() => console.log("Timer"), 0);
```

```
for (let i = 0; i < 1e9; i++) {}
```

Timer runs **after loop**, not immediately.

This is called:

 **Timer Clamping / Scheduling Delay**

setTimeout(0) ≠ Instant Execution

It means:

“Run this after current execution completes.”

Used for:

- ✓ Deferring heavy work
- ✓ Breaking blocking tasks

setInterval() — Run Repeatedly

```
setInterval(() => {  
  console.log("Runs every 1 second");  
}, 1000);
```

Clearing Timers (VERY Important in React / Node)

Timers must be manually stopped.

clearTimeout

```
const id = setTimeout(() => {  
  console.log("Hello");  
}, 2000);
```

```
clearTimeout(id);
```

clearInterval

```
const id = setInterval(() => {  
  console.log("Tick");  
}, 1000);
```

```
clearInterval(id);
```

Real-World Use Cases

Use Case	Use
Debounce input	setTimeout
Retry logic	recursive setTimeout
Clock	setInterval
Polling server	recursive setTimeout
Animation fallback	timers
Cleanup effects	clear timers

How Timers Interact with Event Loop

Timers go to **MACROTASK QUEUE**.

Q. Why does this not print in order? (Event Loop Basics)

```
console.log("A");
```

```
setTimeout(() => {  
  console.log("B");  
}, 0);
```

```
console.log("C");
```

output

A

C

B

Do you understand:

- JS is **non-blocking**
- `setTimeout` goes to **Web API**
- Callback enters **Macrotask Queue**
- Runs only when **Call Stack is empty**



Key Line to Say

“Even with 0ms delay, it waits for the current execution to finish.”

Q. What is the difference between `setTimeout` and `setInterval`?

`setTimeout` is for delayed execution, while `setInterval` is for repeated execution — but `setTimeout` is preferred in production because it avoids execution drift.

Q. Is the delay in `setTimeout(fn, 1000)` guaranteed to be exactly 1 second?

✗ No — It's NOT guaranteed.

It is the **minimum delay**, not exact timing.

If:

- Main thread is busy
- CPU blocked

- Tab inactive

Execution may happen later.

Q. Why is `setInterval` dangerous with async functions?

Because it **does not wait** for previous execution to finish.

Example problem:

API takes 3 seconds

Interval runs every 2 seconds

Now multiple API calls stack → leads to:

- Race conditions
- Server overload
- Memory issues

Correct approach: Use recursive `setTimeout`.

Q. When should you avoid using `setInterval`?

Avoid in:

- API polling
- Async workflows
- Long-running tasks

Use recursive `setTimeout` instead.